



GUÍA VISUAL Y TÉCNICA · SEPTIEMBRE 2026

JEV EN SIMPLE

Cómo diseñar decisiones tipadas, rápidas y utilizables por software

Vanesa Burman · Technical Leader · IA aplicada

ANTES DE EMPEZAR

Qué vas a encontrar en esta guía

01

ENTENDER

Qué es Jev y por qué no es un chatbot.

02

DISEÑAR

Estado, preguntas, primitivas y confianza.

03

APLICAR

Casos de sistemas, soporte, marketing y RR. HH.

04

EVALUAR

Límites, métricas, piloto y despliegue responsable.

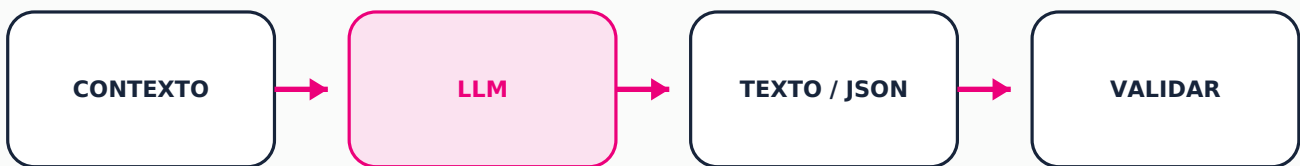
Esta guía explica una tecnología en acceso temprano. Los benchmarks públicos son del proveedor y deben validarse con datos del propio sistema.

LA INTERFAZ IMPORTA

Cuando pedimos texto para obtener una decisión

Un LLM puede devolver JSON. El problema es que su mecanismo central sigue siendo generar texto: después hay que parsear, validar y controlar esa salida.

FLUJO GENERATIVO



Es flexible y útil cuando necesitamos redactar, explicar, sintetizar o razonar sobre una respuesta abierta.

FLUJO DE DECISIÓN TIPADA



La salida ya tiene una forma que el software conoce: una opción, un puntaje o una probabilidad.

La pregunta no es qué IA reemplaza a cuál. Es qué autoridad debe tener cada pieza dentro del sistema.

MAPA RÁPIDO

Jev frente a lo que ya

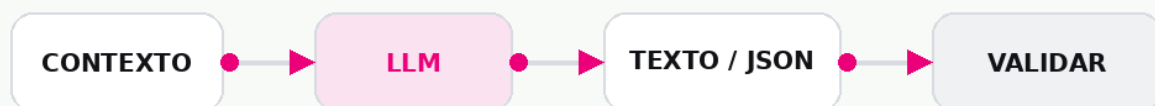
JEV | EN SIMPLE

NO TODA IA TIENE QUE HABLAR

A veces el sistema solo necesita una decisión.

LO QUE YA CONOCEMOS

LLM PARA DECIDIR



Genera una salida que después hay que interpretar y controlar.

OTRA INTERFAZ

JEV

DECISIÓN TIPADA



No inventa el formato.

Puede elegir mal: hay que medirlo.

CHOICE

SCORE

NOUL

CADA PIEZA CON SU AUTORIDAD

JEV

DECIDE

CÓDIGO

EJECUTA

LLM

EXPLICA

PERSONA

VALIDA

No es reemplazo. Es otra pieza.

La pregunta es qué autoridad darle a cada una.

DEFINICIÓN

Jev en una frase

Jev es el primer System One Model de TypeSafe: recibe un estado y preguntas tipadas; devuelve respuestas estructuradas y probabilidades que el código puede usar directamente.

estado + preguntas tipadas → respuestas tipadas + probabilidades

SÍ ES

- Un modelo para juicios semánticos breves y acotados.
- Una interfaz para clasificar, puntuar, verificar y enrutar.
- Una pieza que expone incertidumbre mediante probabilidades.

NO ES

- Un chatbot o generador de texto libre.
- Un agente que decide su próximo paso.
- Un reemplazo de reglas, cálculos o permisos.
- Una garantía de que la decisión sea correcta.

CRITERIO PRÁCTICO

Si la salida debe ser leída por una persona, probablemente necesitás un LLM. Si debe activar una rama conocida del sistema, puede ser un caso para Jev. Si la respuesta ya puede escribirse exactamente, usá código.

UNA LLAMADA

Tres partes: estado, pregunta y respuesta

La calidad no depende de un prompt largo. Depende de representar bien los hechos, delimitar el juicio y definir una salida que el código pueda consumir.

1

ESTADO

Hechos y contexto relevantes. Puede ser texto o JSON. Debe ser pequeño, explícito y suficiente.

2

PREGUNTAS

Cada pregunta pide un solo juicio. Varias preguntas independientes pueden viajar juntas.

3

RESPUESTAS

Valores tipados, distribuciones y confianza. El código decide qué hacer con ellos.

EJEMPLO MÍNIMO

```
state = {  
  "mensaje": "Me cobraron dos veces. Quiero el reintegro"  
}  
  
question = Choice(  
  instructions="¿Cuál es el pedido principal?",  
  criteria={"reintegro": "...", "soporte": "...", "otro": "..."}  
)  
  
answer → choice: "reintegro" + probabilities + confidence
```

El ID de la pregunta es para el código. La instrucción debe contener la pregunta completa.

BLOQUES DE CONSTRUCCIÓN

Choice, Score y Noul

Las tres primitivas no son estilos de respuesta. Son contratos distintos entre el modelo y el software.

CHOICE

¿Cuál de estas opciones?

Devuelve una opción, distribución y confianza.

EJEMPLO Enrutar un ticket a facturación, soporte u otro.

SCORE

¿En qué nivel?

Devuelve una posición sobre niveles ordenados, distribución y confianza.

EJEMPLO Graduar severidad: baja, media, alta, crítica.

NOUL

¿Es verdadera esta condición?

Devuelve la probabilidad de sí entre 0 y 1.

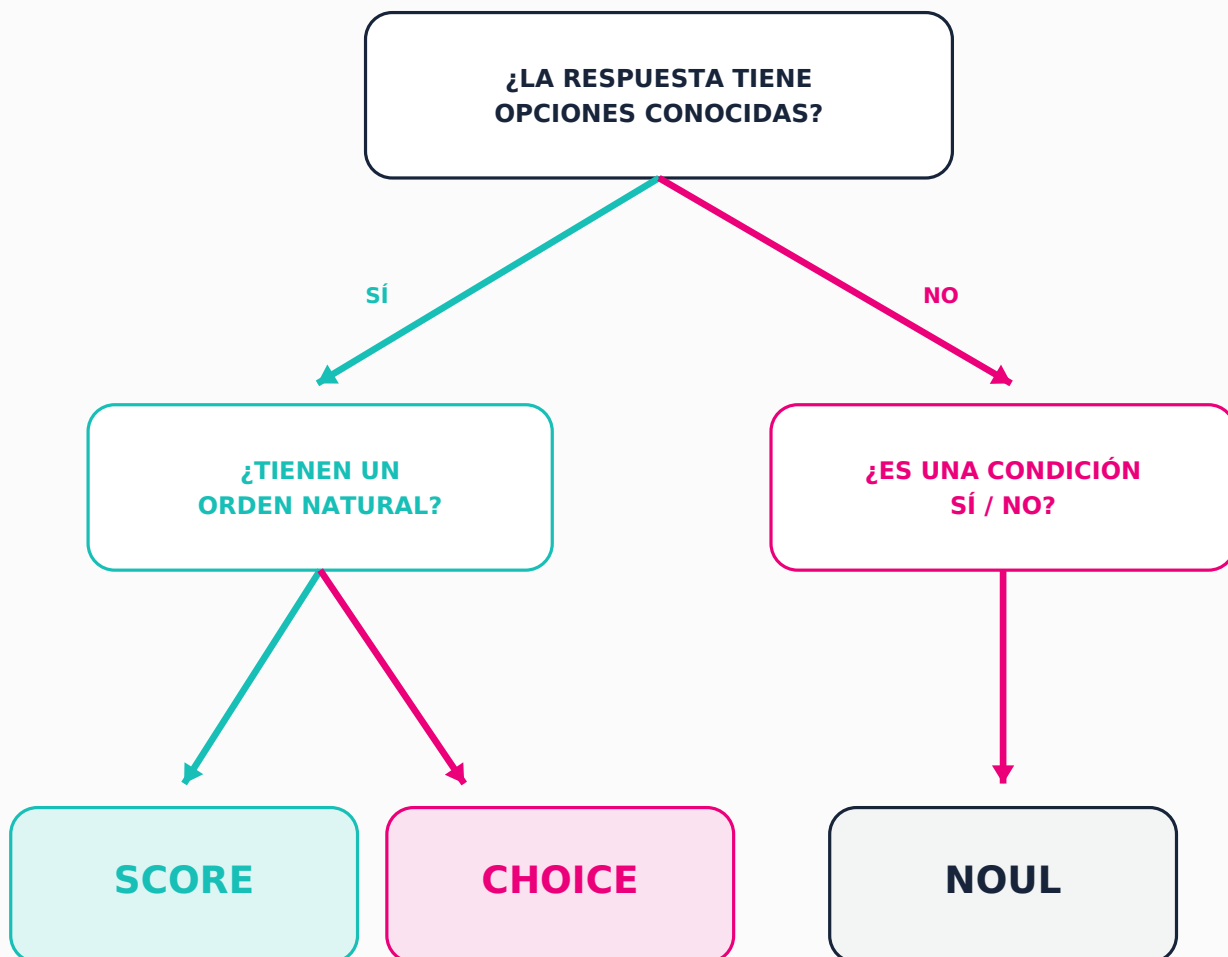
EJEMPLO ¿El cliente está pidiendo un reintegro?

Agregar “otro” o “ninguna opción” evita forzar un caso nuevo dentro de una taxonomía incompleta.

ÁRBOL DE DECISIÓN

Cómo elegir la primitiva

Empezá por la forma exacta que necesita la siguiente línea de código.

**SI NINGUNA ENCAJA**

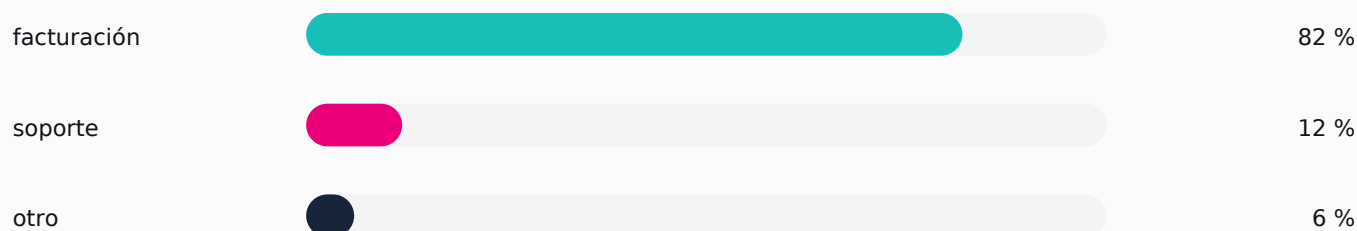
Puede que la tarea necesite razonamiento, generación o una regla determinística. No conviertas una pregunta compleja en Choice solo porque el sistema necesita una etiqueta.

PROBABILIDAD Y CONFIANZA

No son decoración: gobiernan el flujo

Choice y Score entregan una distribución completa y un indicador de confianza. Noul entrega directamente la probabilidad de sí.

DISTRIBUCIÓN CONCENTRADA



Alta confianza no convierte una decisión en verdad. Solo indica que la distribución está concentrada.

TRES SALIDAS OPERATIVAS

AUTOMATIZAR

Alta confianza + acción reversible

CONFIRMAR

Zona intermedia o dato faltante

ESCALAR

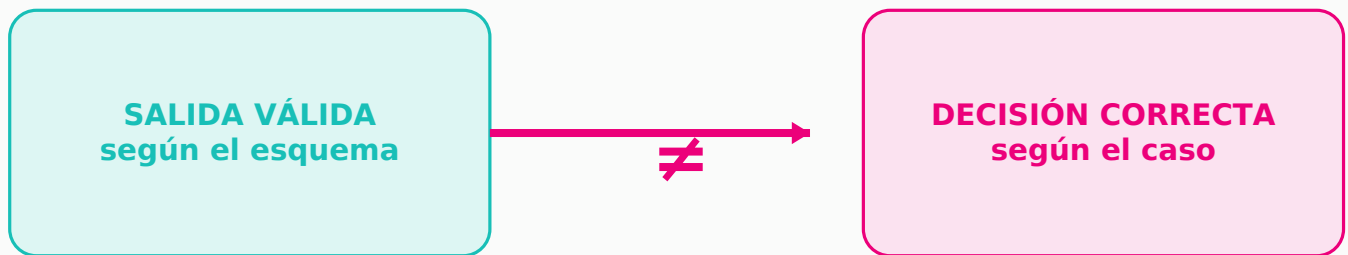
Baja confianza o alto impacto

Los umbrales deben calibrarse por decisión y por nivel de riesgo. Un único “0,8” global suele esconder diferencias importantes entre clases.

LA ACLARACIÓN IMPORTANTE

“Cero alucinaciones” no significa “cero errores”

Jev no puede devolver un valor fuera del contrato. Eso elimina errores de forma; no garantiza que el juicio elegido sea correcto.



GARANTÍA ESTRUCTURAL

- El valor pertenece a las opciones definidas.
- El formato es utilizable por código.
- No hay que recuperar la decisión desde prosa generada.

CALIDAD DEL JUICIO

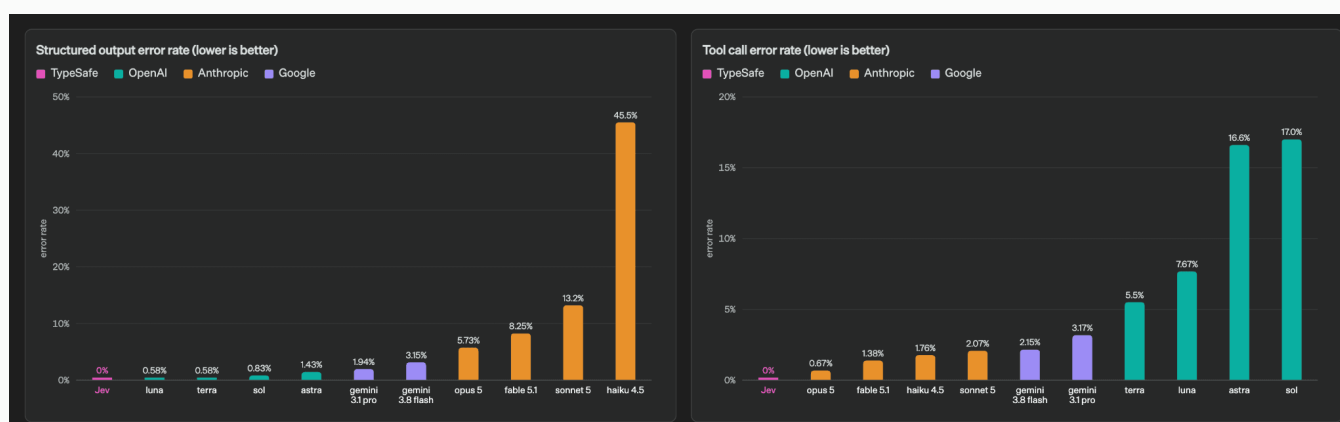
- Puede elegir una opción equivocada.
- Puede estar mal calibrado para un dominio.
- Necesita evaluación con casos reales y revisión de errores.

Type safety es una condición valiosa para automatizar. No es una prueba de exactitud, justicia ni conveniencia.

GRÁFICO DEL PROVEEDOR

Errores estructurados y llamadas a herramientas

TypeSafe muestra una tasa estructural de 0 % para Jev porque el esquema está garantizado por diseño. El propio proveedor aclara que ese valor no es empírico.



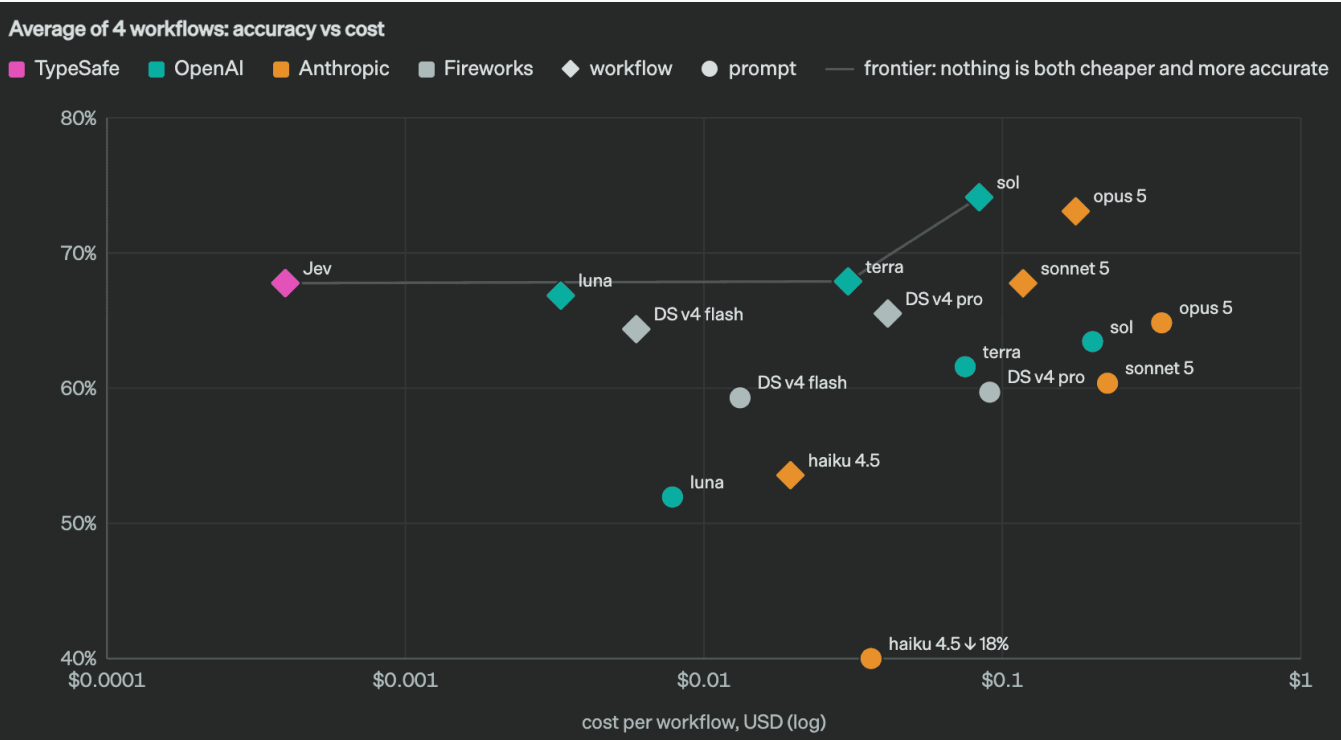
CÓMO LEERLO

El gráfico sirve para comparar errores de contrato, no la corrección semántica de cada decisión. Para evaluar un sistema real todavía hacen falta precisión por clase, calibración, cobertura automática y análisis de desacuerdos.

BENCHMARK DEL PROVEEDOR

Precisión frente a costo por workflow

El lanzamiento ubica a Jev en la frontera costo/precisión para cuatro workflows internos. Es evidencia interesante, pero no sustituye un benchmark propio.



70-500 ms

latencia publicada por TypeSafe

USD 0,042

por millón de tokens de entrada

Early access

estado del producto al publicar esta guía

La empresa reconoce que sus mayores ratios son el extremo alto de las pruebas, que los workflows fueron construidos por integrantes del equipo y que parte de la referencia proviene de modelos externos.

Fuente: TypeSafe AI, lanzamiento de System One y Jev. Costos, latencias y resultados vigentes en septiembre de 2026; pueden cambiar.

PATRÓN DE REFERENCIA

Jev decide; el código gobierna

La arquitectura robusta distribuye autoridad. Ningún componente debería concentrar juicio, reglas, explicación y efectos irreversibles.

JEV

Clasifica · puntúa · verifica

Entrega señales tipadas**CÓDIGO**

Reglas · permisos · umbrales

Controla y ejecuta**LLM**

Razona · redacta · explica

Trabaja con respuestas abiertas**PERSONA**

Excepciones · riesgo · contexto

Aprueba o corrige

Una señal de Jev no debería producir directamente un efecto irreversible. El código conserva permisos, auditoría y mecanismos de rollback.

Diseñar una decisión en seis pasos

La integración técnica es corta. El trabajo real está en la taxonomía, los criterios, los casos frontera y la evaluación.

- 1 DECISIÓN** Definir qué rama necesita elegir el sistema.
- 2 ESTADO** Seleccionar solo hechos relevantes y disponibles.
- 3 CONTRATO** Elegir Choice, Score o Noul.
- 4 CRITERIOS** Describir opciones y límites sin ambigüedad.
- 5 EVALUACIÓN** Construir casos representativos y medir.
- 6 AUTORIDAD** Fijar umbrales, revisión y acciones permitidas.

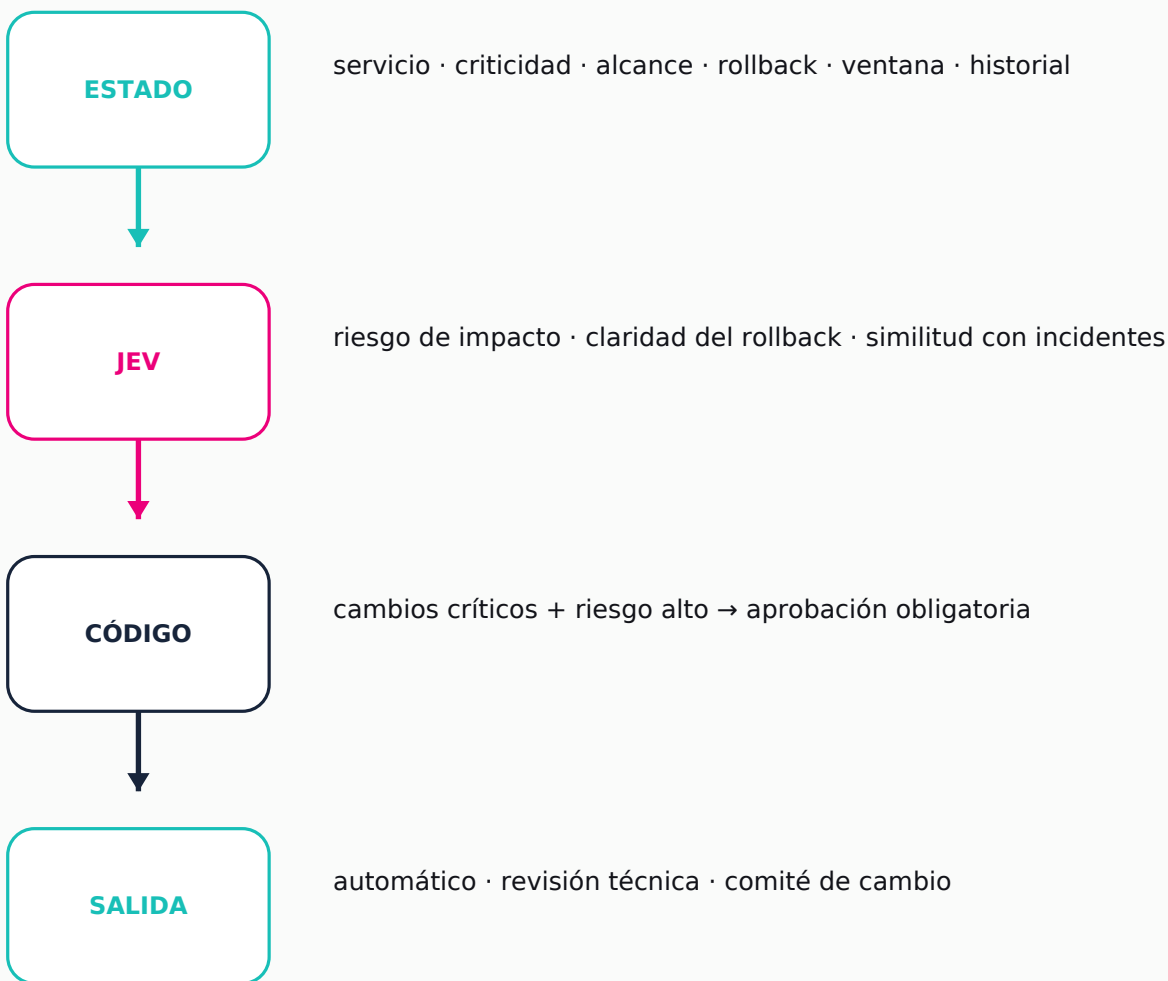
REGLA DE ORO

Una pregunta debe representar el tipo de juicio que una persona experta podría hacer rápidamente con el contexto correcto. Si requiere análisis prolongado, conviene descomponerla.

GESTIÓN DE CAMBIOS

Qué despliegues necesitan revisión adicional

El objetivo no es permitir o bloquear automáticamente. Es reunir señales consistentes para que el pipeline aplique una política explícita.



PREGUNTAS CANDIDATAS

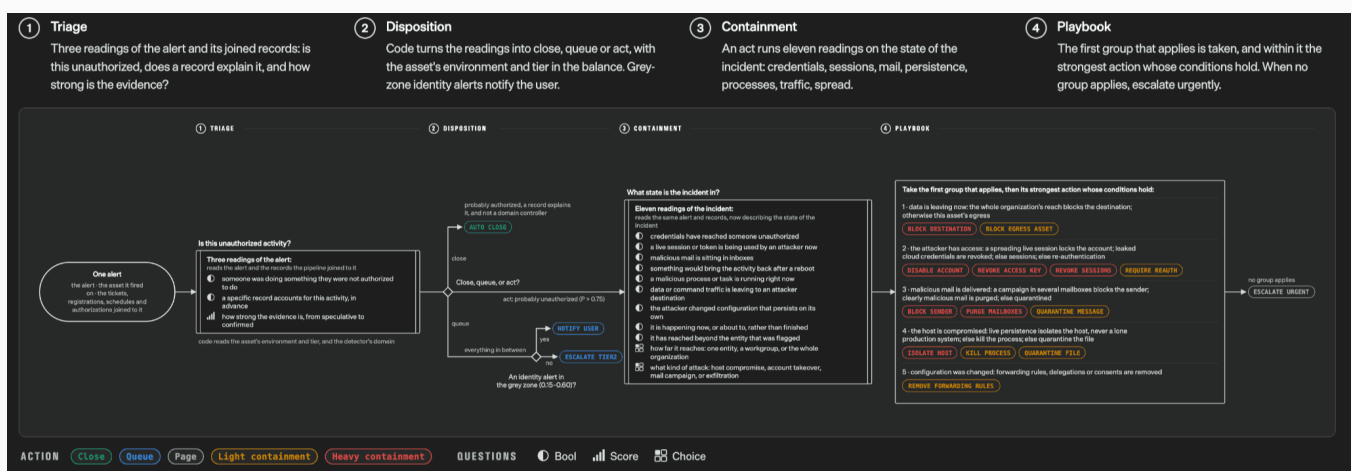
- Choice: ¿qué tipo de cambio domina? configuración, datos, infraestructura, aplicación u otro.
- Score: ¿qué nivel de impacto operativo puede producir?
- Noul: ¿el plan de rollback es suficientemente explícito para ejecutar?

Mantener en código: permisos, ambientes habilitados, horario, tests obligatorios y bloqueo de acciones irreversibles.

TRIAGE DE INCIDENTES

Clasificar antes de escalar

Un buen flujo separa intención, severidad, evidencia y riesgo. Luego el código combina señales y elige el playbook permitido.



VERSIÓN APLICABLE A SOPORTE

Ticket + logs + servicio afectado → triage tipado → severidad y confianza → regla de escalamiento → playbook. Las acciones sobre infraestructura siguen bajo permisos y auditoría.

QUÉ MEDIR

- Precisión por categoría y severidad.
- Tiempo hasta la primera acción útil.
- Incidentes que se automatizaron incorrectamente.
- Volumen escalado y causas de baja confianza.

Imagen oficial: workflow de seguridad publicado por TypeSafe AI. Se incluye como ejemplo de descomposición; el caso productivo debe rediseñarse para cada organización.

CONTROL Y ENRUTAMIENTO DE CAMPAÑAS

Decidir sin pedirle al modelo que escriba todo

Jev puede asistir en clasificación, chequeos y riesgo; el LLM puede generar variantes; una persona conserva la aprobación editorial.



EJEMPLOS DE PREGUNTAS

- Choice: ¿cuál es el objetivo dominante? awareness, consideración, conversión o retención.
- Score: ¿qué nivel de riesgo de marca presenta esta pieza?
- Noul: ¿incluye una afirmación que requiere evidencia o revisión legal?

LÍMITE IMPORTANTE

Una clasificación no prueba que una pieza sea verdadera, legal o culturalmente apropiada. Los claims, derechos, segmentos sensibles y reputación requieren controles específicos.

ORIENTAR SIN AUTOMATIZAR DECISIONES SENSIBLES

Asistencia para organizar, no para excluir

En RR. HH. conviene limitar Jev a señales descriptivas y recorridos de revisión. La decisión de contratación, desempeño o sanción necesita supervisión humana y criterios auditables.

USO RAZONABLE

- Enrutar consultas a la política correcta.
- Detectar información faltante en una postulación.
- Clasificar el tema principal de una encuesta abierta.
- Priorizar revisión manual por urgencia explícita.

NO DELEGAR

- Descartar personas por ajuste cultural.
- Inferir rasgos, salud o potencial.
- Asignar desempeño sin evidencia.
- Tomar decisiones laborales de alto impacto.

El sistema debe registrar criterios, versiones, correcciones y motivos de escalamiento. La revisión de sesgos se hace sobre resultados reales, no solo sobre el prompt.

EJEMPLO SEGURO

Noul: ¿el documento menciona experiencia laboral con Python? No pregunta si la persona “es buena”, sino si existe evidencia explícita de un hecho definido.

PATRONES REUTILIZABLES

Dónde encaja mejor una decisión tipada

Los patrones ayudan a identificar oportunidades sin convertir cada problema en una llamada al modelo.

ROUTER

Elige un área, flujo o especialista.

`ticket → facturación / soporte / otro`

CONFIDENCE GATE

Decide actuar, confirmar o escalar.

`baja confianza → revisión`

FAN-OUT

Evalúa varias dimensiones en paralelo.

`intención + urgencia + riesgo`

VERIFIER

Juzga una condición sobre una salida.

`¿la respuesta está respaldada?`

GUARDRAIL

Clasifica entradas o resultados riesgosos.

`contenido adversarial o sensible`

COMPOSITE SCORE

Combina juicios atómicos en código.

`prioridad = impacto + urgencia`

Si la salida puede escribirse exactamente con una regla, el patrón correcto sigue siendo código.

JAGGEDNESS

Dónde Jev puede fallar

La documentación oficial de Jev 1.13 publica límites concretos. Diseñar alrededor de ellos es parte del trabajo.

LÍMITE	RIESGO	DISEÑO
Lectura literal	Puede responder lo escrito y no la intención.	Explicitar límites y casos frontera.
Cálculo y conteo	Precisión numérica débil.	Calcular en código.
Fechas	Comparaciones poco fiables.	Extraer y resolver en código.
Indirección	Pierde precisión con varios saltos.	Nombrar el campo relevante.
Estado grande	El detalle irrelevante distrae.	Filtrar antes de enviar.
Adversarial	El texto puede orientar la clasificación.	Probar ataques y mantener guardas.
Invariantes	Respuestas equivalentes pueden divergir.	Validar coherencia en código.
Generación	No produce texto libre.	Usar un LLM.

No ocultes los límites con más instrucciones. Cuando el problema no tiene forma System One, cambiá la arquitectura.

MEDIR ANTES DE AUTOMATIZAR

Qué métricas importan

La exactitud promedio no alcanza. Un sistema útil debe mostrar dónde se equivoca, cuándo sabe que no sabe y qué costo operativo deja.

Precisión por opción

¿En qué clases se equivoca?

Calibración

Cuando dice 0,8, ¿acierta cerca del 80 %?

Cobertura automática

¿Qué proporción puede resolverse sin revisión?

Falsos automatismos

¿Cuántos casos actuaron cuando no correspondía?

Tasa de escalamiento

¿Cuánto trabajo humano conserva?

Tiempo hasta acción

¿Mejora la experiencia completa?

Correcciones humanas

¿Qué datos o criterios faltan?

El conjunto de evaluación debe incluir casos frecuentes, ambiguos, raros, adversariales y de alto impacto.

DESPLEGAR POR ETAPAS

Un piloto de siete movimientos

Empezá con una decisión repetitiva, estrecha y reversible. El objetivo inicial es aprender, no maximizar automatización.

- 1 Elegir una decisión relevante y acotada.
- 2 Reunir entre 100 y 300 casos representativos.
- 3 Definir opciones, niveles y categorías de escape.
- 4 Construir una línea base con reglas o LLM estructurado.
- 5 Probar Jev sobre el mismo conjunto.
- 6 Ejecutar en modo sombra: recomienda, pero no actúa.
- 7 Ajustar criterios y liberar solo acciones reversibles.

CRITERIO DE SALIDA

No pasar a producción si no hay evaluación representativa, trazabilidad, umbrales por riesgo y un camino claro para revisar o revertir decisiones.

CHECKLIST DE ARQUITECTURA

Antes de integrar Jev

Doce preguntas para separar entusiasmo de una decisión de producto defendible.

☐ ¿La salida es una opción, nivel o probabilidad conocida?

☐ ¿Cada pregunta pide un solo juicio?

☐ ¿El estado contiene solo datos relevantes?

☐ ¿Existe una salida “otro”, “no consta” o revisión?

☐ ¿Las reglas exactas permanecen en código?

☐ ¿Los permisos y efectos están fuera del modelo?

☐ ¿Hay casos representativos para evaluar?

☐ ¿Se mide por clase y no solo en promedio?

☐ ¿Los umbrales cambian según riesgo?

☐ ¿Los casos inciertos se escalan?

☐ ¿Las decisiones quedan auditadas?

☐ ¿Podemos reemplazar el modelo sin rediseñar todo?

FUENTES PRINCIPALES

- docs.typesafe.ai/introduction · primitives · confidence
- docs.typesafe.ai/model-jaggedness/jev-1.13
- typesafe.ai/blog/introducing-system-one-models-and-jev

La tecnología es una pieza. El sistema es la decisión.

Soy Vanesa Burman, Technical Leader y especialista en diseño de sistemas de trabajo e IA aplicada. En HACIBLE convierto problemas reales —con excepciones, personas y restricciones— en soluciones que puedan probarse, explicarse y mejorarse.

¿Qué podemos trabajar?

- Mapa de decisiones y fricciones del sistema.
- Prototipos con IA, reglas y revisión humana.
- Evaluación, métricas y estrategia de despliegue.
- Acompañamiento técnico y de producto para equipos.



CONVERSEMOS

Contame qué decisión repetitiva está frenando hoy a tu equipo. Diseñemos un piloto concreto, medible y con la autoridad bien distribuida.

hacible-jev.vanesa-burman.chatgpt.site

No adapto personas a sistemas ideales.

Diseño sistemas para que personas reales puedan avanzar.